

ЛАБОРАТОРНАЯ РАБОТА №5

«ИССЛЕДОВАНИЕ АЛГОРИТМОВ ПРОГРАММИРОВАНИЯ ДЛЯ ВЫПОЛНЕНИЯ ОПЕРАЦИЙ НАД НЕЛИНЕЙНЫМИ СТРУКТУРАМИ ДАННЫХ НА ЯЗЫКЕ C/C++»

1. Цель работы

Изучение нелинейных структур данных и приобретение навыков разработки и отладки программ, использующих древовидные структуры данных. Исследование особенностей работы с поисковыми бинарными деревьями на языке C/C++.

2. Постановка задачи и вариант задания

Структуру данных из предыдущей лабораторной работы использовать в качестве информационного поля бинарного дерева. Написать функции:

- организации дерева,
- просмотра (обхода) дерева,
- отображения структуры дерева;
- добавления узла в дерево,
- исключения узла из дерева,
- сохранения в файл,
- загрузки данных из файла и создания по этим данным нового дерева,
- освобождения динамической памяти, занимаемой деревом (обязательно вызывать при выходе из программы),
- выполнения действий в соответствии с вариантом, приведенным ниже.

Значения и количество записей в таблице пользователь выбирает самостоятельно (т.е. количество строк таблицы не задается, память под узлы дерева выделяется динамически). Работу программы необходимо оформить в виде меню.

Вариант 4: Функция, которая определяет уровень, на котором находится самый правый узел непустого дерева. Функция печати ФИО ученых, у которых количество опубликованных статей больше 100, а если таких нет, то вывести соответствующее сообщение.

ХОД РАБОТЫ

3. Структурная схема алгоритма

Структурная схема алгоритма, а также всех функций представлены на рисунках 5.1 – 5.11:

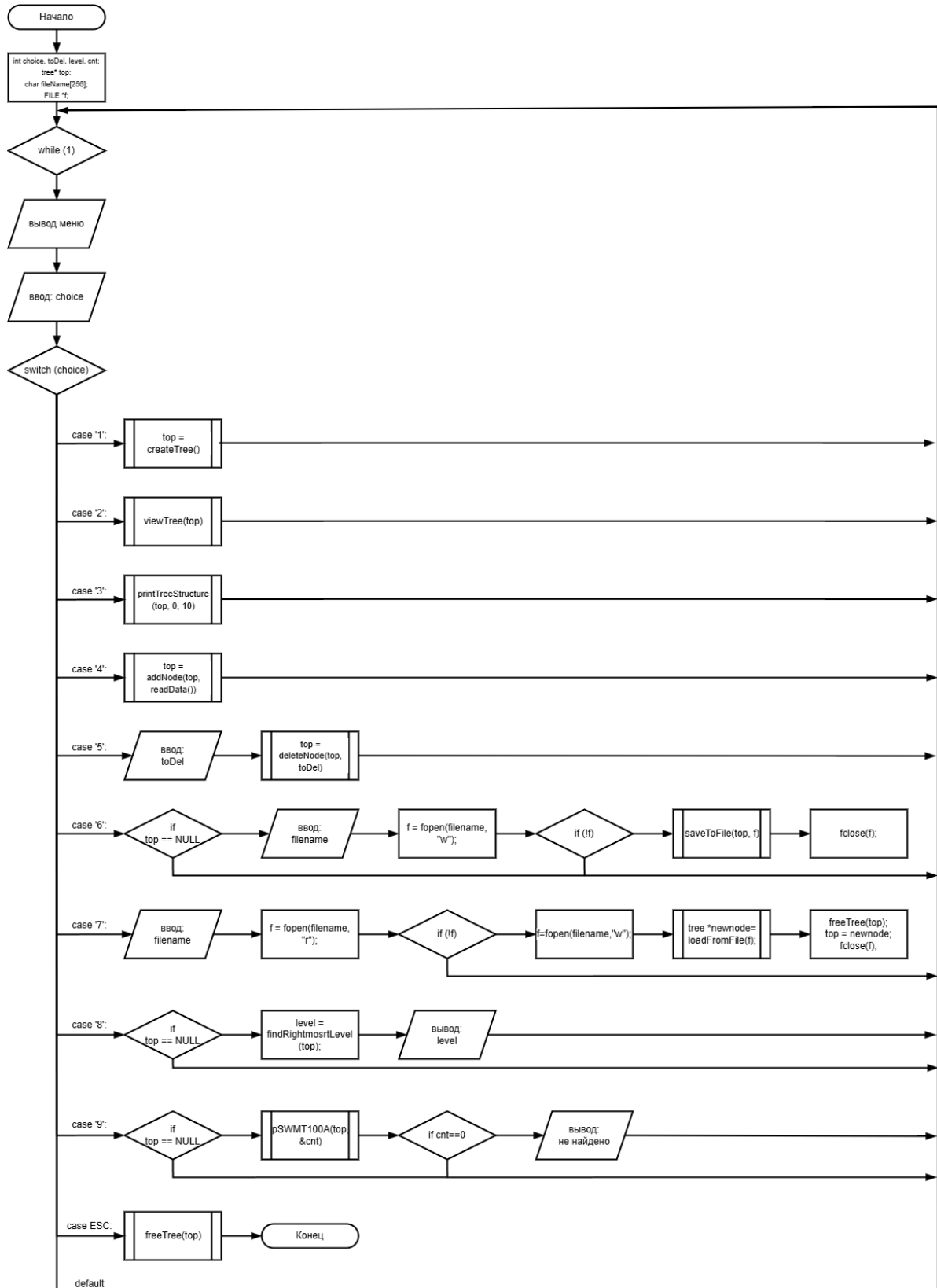


Рисунок 5.1 – Структурная схема функции main()

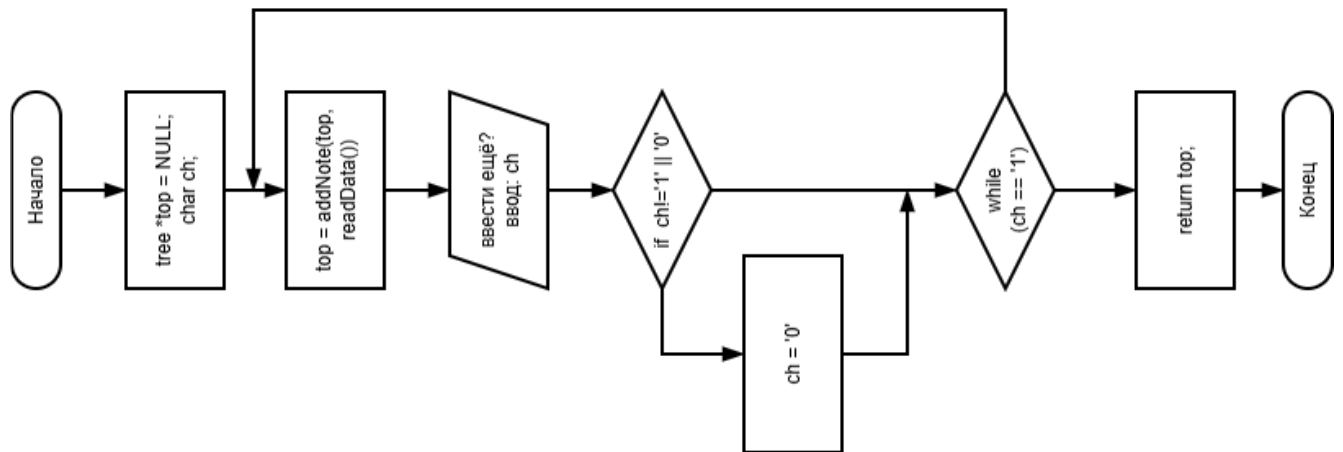


Рисунок 5.2 – Структурная схема функции createTree()

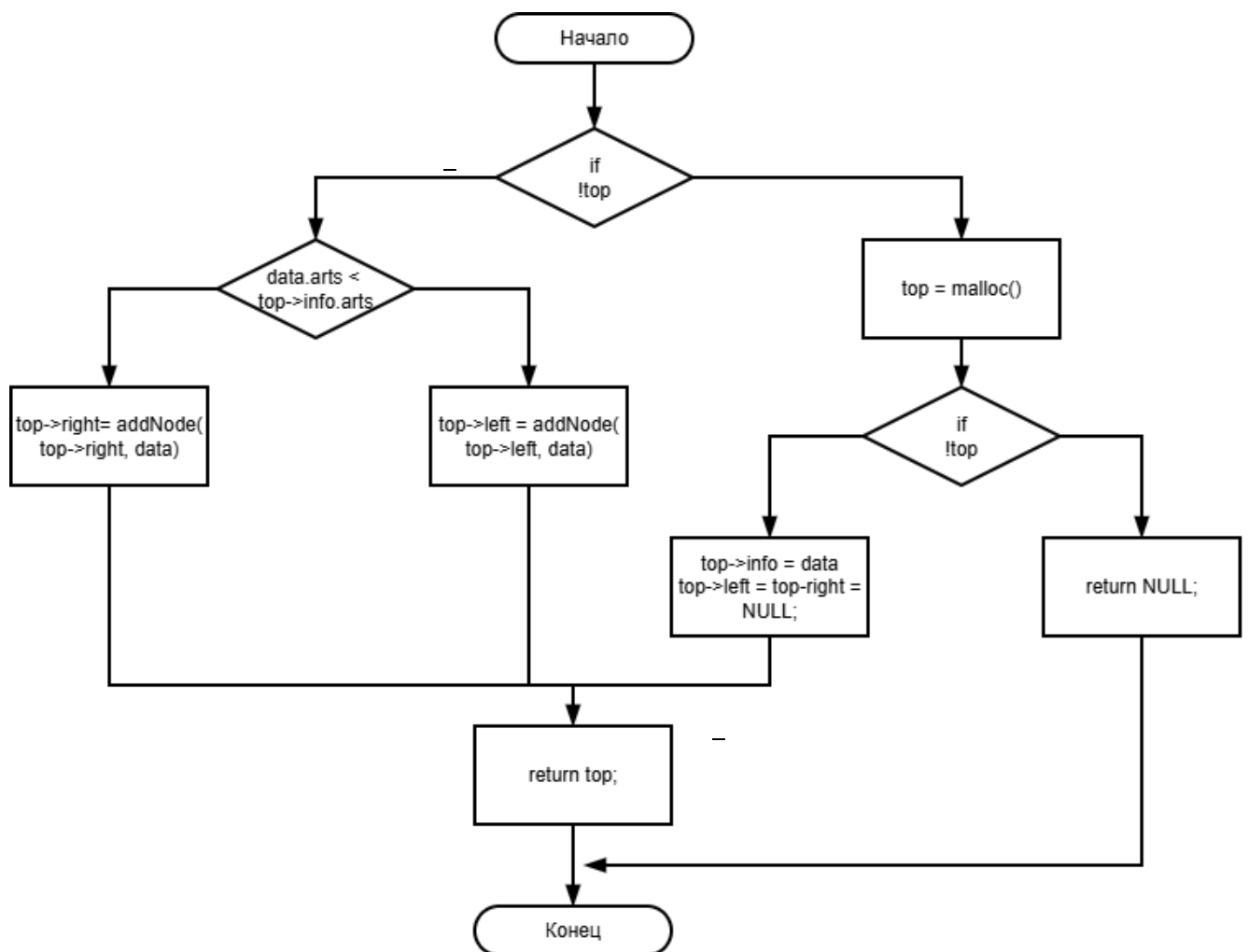


Рисунок 5.3 – Структурная схема функции addNode()

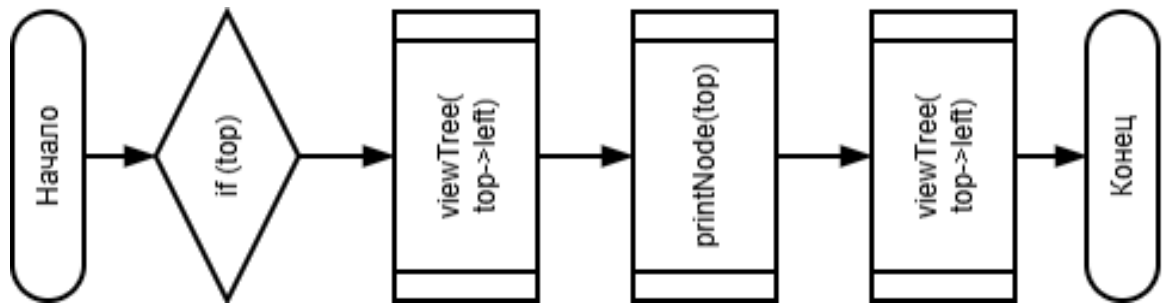


Рисунок 5.4 – Структурная схема функции viewTree()

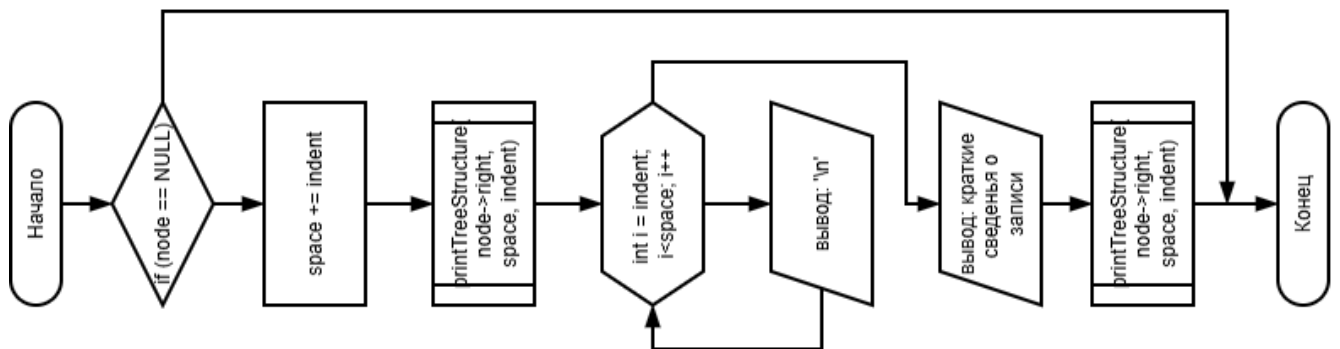


Рисунок 5.5 – Структурная схема функции printTreeStructure()

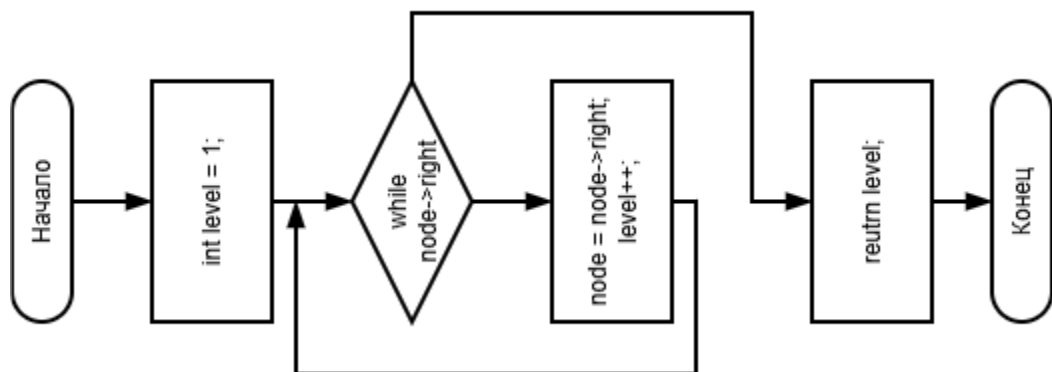


Рисунок 5.6 – Структурная схема функции findRightmostLevel()

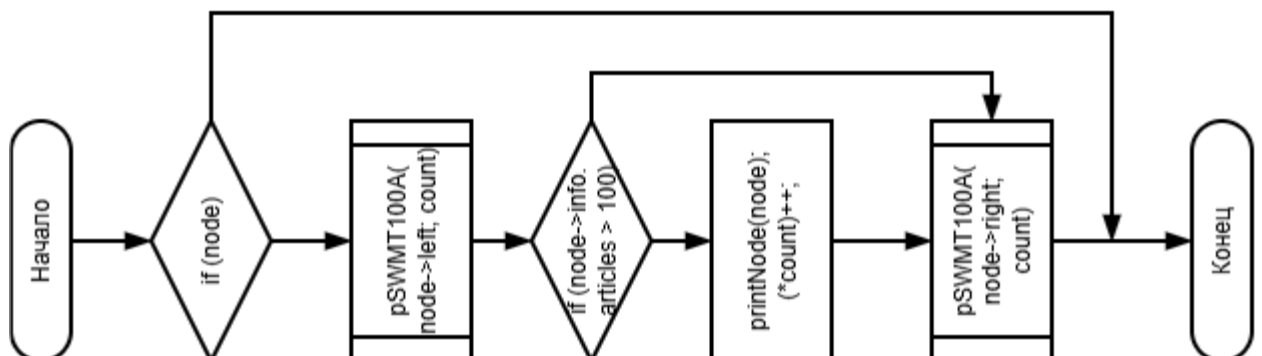


Рисунок 5.7 – Структурная схема функции printScientistsWithMore100Articles()

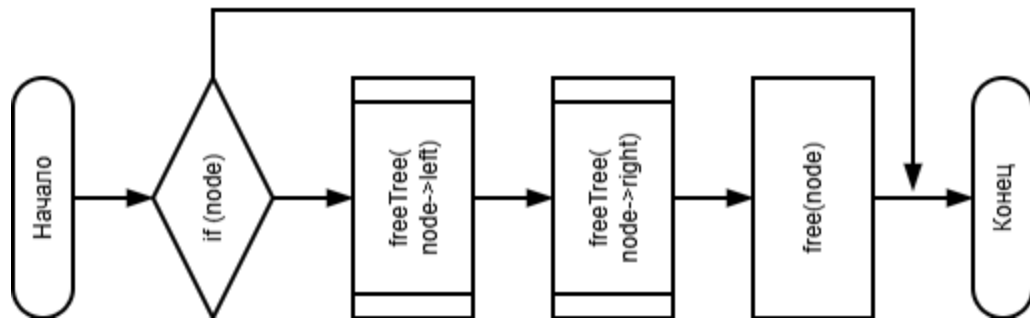


Рисунок 5.8 – Структурная схема функции `printScientistsWithMore100Articles()`

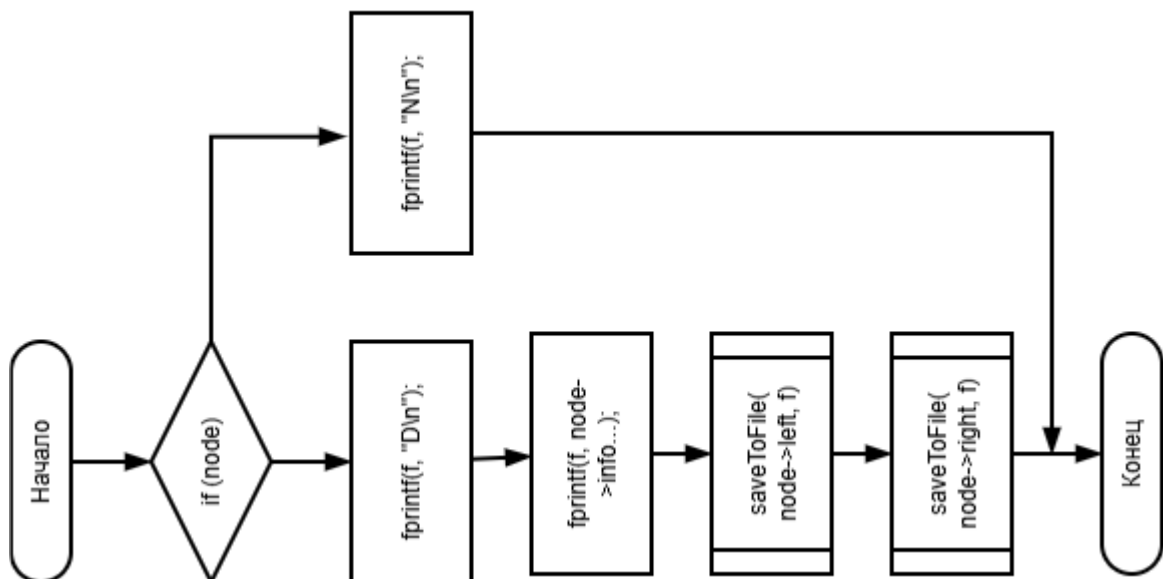


Рисунок 5.9 – Структурная схема функции `saveToFile()`

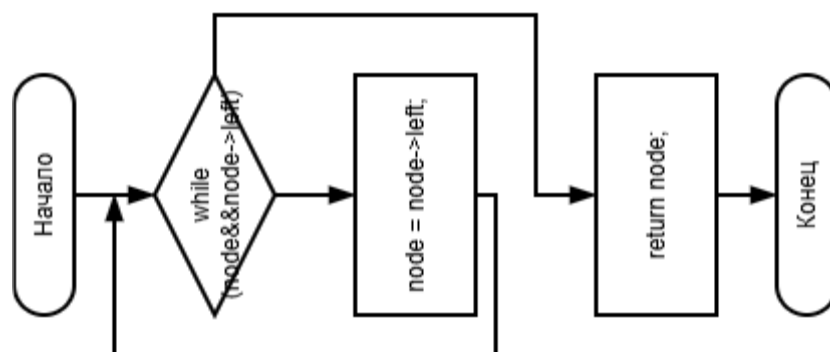


Рисунок 5.10 – Структурная схема функции `findMin()`

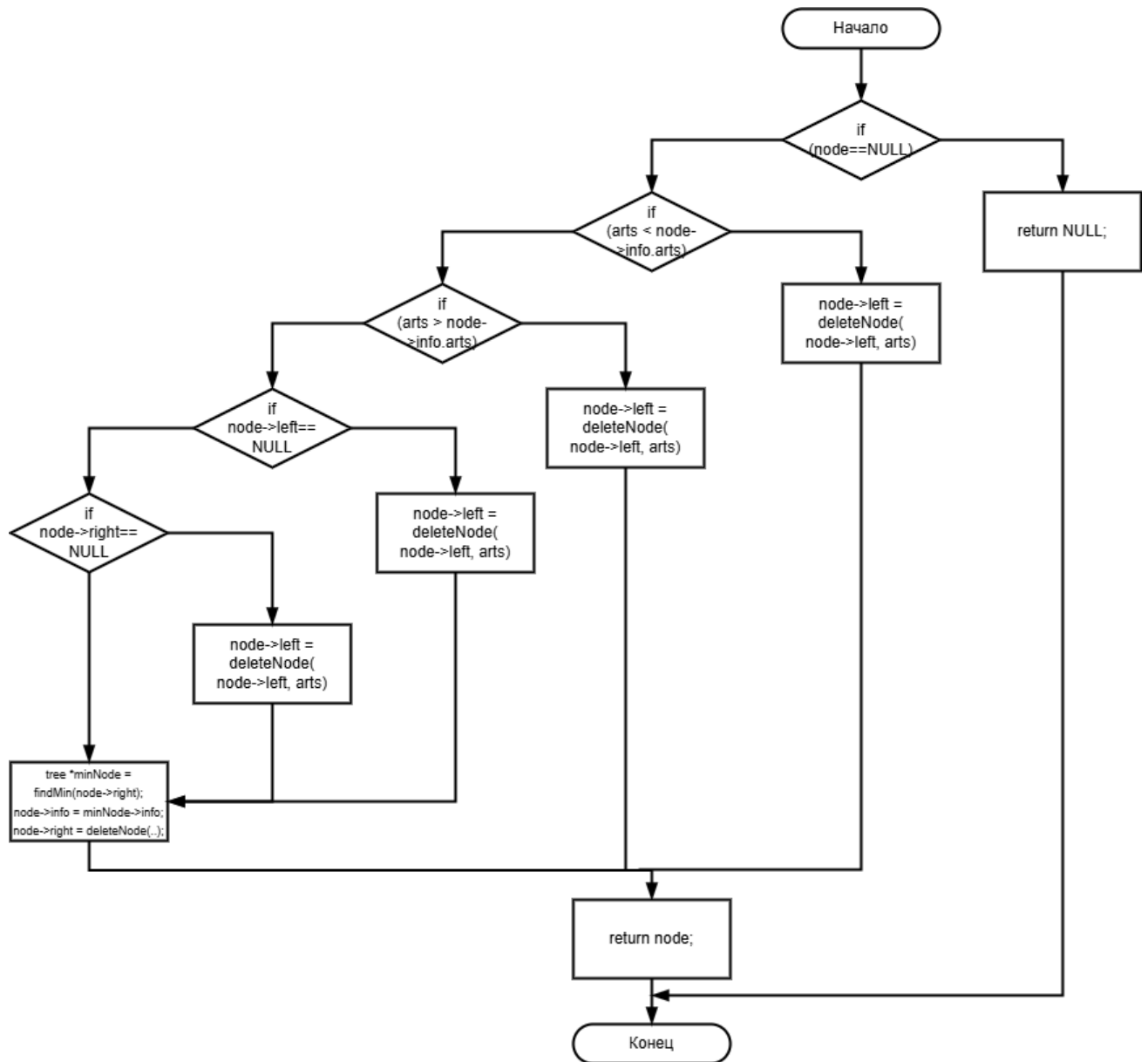


Рисунок 5.11 – Структурная схема функции deleteNode()

4. Текст кода на языке C. Тестирование и отладка программы

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
#include <conio.h>
#define ESC 27

struct scientist {
    int id;
    char name[76];
    char area[26];
    char degree[26];
    int articles;
    int quotes;
    int hirshIndex;
};

struct tree {
    struct scientist info;
    struct tree *left, *right;
};

// Прототипы
struct scientist readData();
struct tree *createTree();
struct tree *addNode(struct tree *, struct scientist);
void viewTree(struct tree *);
void printTreeStructure(struct tree *, int, int);
struct tree *deleteNode(struct tree *, int );
struct tree *findMin(struct tree *);
void saveToFile(struct tree *, FILE *);
struct tree *loadFromFile(FILE *);
int findRightmostLevel(struct tree *);
void printScientistsWithMoreThan100Articles(struct tree *, int *);
void freeTree(struct tree *);

// Вспомогательные
void clearInputBuffer();
void printTableHeader();
void printNode(struct tree *);

int main() {
    SetConsoleCP(65001);
    SetConsoleOutputCP(65001);
    struct tree *top = NULL;
    int choice;
    char filename[256];
    FILE *f;
    int cnt, level, toDel;

    while (1) {
        system("cls");
        printf("===== МЕНЮ =====\n");
        printf("1. Организация дерева\n");
        printf("2. Просмотр дерева (обход)\n");
        printf("3. Отображение структуры дерева\n");
        printf("4. Добавить узел дерева\n");
        printf("5. Исключить узел из дерева\n");
        printf("6. Сохранить дерево в файл\n");
        printf("7. Загрузить дерево из файла\n");
        printf("8. Определить уровень самого правого узла\n");
        printf("9. Вывести учёных с более чем 100 статей\n");
```

```

printf("ESC. Выход\n");
printf("-> ");
choice = getch();

switch (choice) {
case '1':
    system("cls");
    top = createTree();
    break;

case '2':
    system("cls");
    printTableHeader();
    viewTree(top);
    printf("\nНажмите любую клавишу...");
    getch();
    break;

case '3':
    system("cls");
    printf("\nСтруктура дерева (повёрнута влево):\n");
    printTreeStructure(top, 0, 10);
    printf("\nНажмите любую клавишу...");
    getch();
    break;

case '4':
    system("cls");
    top = addNode(top, readData());
    printf("Узел добавлен.\nНажмите любую клавишу...");
    getch();
    break;

case '5':
    system("cls");
    printf("Введите ID удаляемого учёного: ");
    scanf("%d", &toDel);
    clearInputBuffer();
    top = deleteNode(top, toDel);
    printf("Нажмите любую клавишу...");
    getch();
    break;

case '6':
    if (top == NULL) {
        printf("Дерево пусто, сохранять нечего.\n");
        Sleep(1500);
        break;
    }
    printf("Введите имя файла для сохранения: ");
    fgets(filename, 256, stdin);
    filename[strcspn(filename, "\n")] = '\0';
    f = fopen(filename, "w");
    if (!f) {
        printf("Ошибка открытия файла для записи.\n");
        Sleep(1500);
        break;
    }
    saveToFile(top, f);
    fclose(f);
    printf("Дерево сохранено в файл %s\n", filename);
    printf("Нажмите любую клавишу...");
    getch();
    break;
}

```



```

case '7':
    system("cls");
    printf("Введите имя файла для загрузки: ");
    fgets(filename, 256, stdin);
    filename[strlen(filename)] = '\0';
    f = fopen(filename, "r");
    if (!f) {
        printf("Ошибка открытия файла для чтения.\n");
        Sleep(1500);
        break;
    }
    struct tree *newnode = loadFromFile(f);
    fclose(f);
    if (newnode) {
        freeTree(top);
        top = newnode;
        printf("Дерево успешно загружено из файла %s\n", filename);
    } else {
        printf("Файл пуст или повреждён.\n");
    }
    printf("Нажмите любую клавишу...");
    getch();
    break;

case '8':
    system("cls");
    if (top == NULL) {
        printf("Дерево пусто.\n");
        Sleep(1500);
        break;
    }
    level = findRightmostLevel(top);
    printf("Самый правый узел находится на уровне: %d\n", level);
    printf("Нажмите любую клавишу...");
    getch();
    break;

case '9':
    system("cls");
    if (top == NULL) {
        printf("Дерево пусто.\n");
        Sleep(1500);
        break;
    }
    cnt = 0;
    printf("\nУчёные с количеством статей > 100:\n");
    printScientistsWithMoreThan100Articles(top, &cnt);
    if (cnt == 0) {
        printf("Нет учёных с количеством статей более 100.\n");
    }
    printf("Нажмите любую клавишу...");
    getch();
    break;

case ESC:
    freeTree(top);
    return 0;

default:
    printf("Команда не распознана!\n");
    Sleep(1500);
    break;
}

```

```

    }

void printTableHeader() {
    printf("+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----\n");
    printf("| ID | Фамилия Имя Отчество | Учёная Степень | Область Науки | Кол-во статей | Кол-во цитирований | Индекс Хирша |\n");
    printf("+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----\n");
}

void printNode(struct tree *node) {
    printf("| %-4d | %-75s | %-25s | %-25s | %-13d | %-18d | %-12d |\n",
        node->info.id,
        node->info.name,
        node->info.degree,
        node->info.area,
        node->info.articles,
        node->info.quotes,
        node->info.hirshIndex);
    printf("+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----\n");
}

// Очистка буфера ввода
void clearInputBuffer() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

// Функция ввода данных с корректной очисткой буфера
struct scientist readData() {
    struct scientist data;
    static int nextID = 1;
    data.id = nextID++;

    printf("Введите ФИО: ");
    fgets(data.name, sizeof(data.name), stdin);
    data.name[strcspn(data.name, "\n")] = '\0';
    // Если пользователь ввел пустую строку — повторить
    while (strlen(data.name) == 0) {
        printf("ФИО не может быть пустым. Введите ФИО: ");
        fgets(data.name, sizeof(data.name), stdin);
        data.name[strcspn(data.name, "\n")] = '\0';
    }

    printf("Введите научную область: ");
    fgets(data.area, sizeof(data.area), stdin);
    data.area[strcspn(data.area, "\n")] = '\0';

    printf("Введите учёную степень: ");
    fgets(data.degree, sizeof(data.degree), stdin);
    data.degree[strcspn(data.degree, "\n")] = '\0';

    printf("Введите количество научных статей: ");
    scanf("%d", &data.articles);
    printf("Введите количество цитирований: ");
    scanf("%d", &data.quotes);
    printf("Введите индекс Хирша: ");

```

```

scanf("%d", &data.hirshIndex);
clearInputBuffer(); // очищаем буфер после scanf

printf("\n");
return data;
}

struct tree *createTree() {
    struct tree *top = NULL;
    char ch;
    do {
        top = addNode(top, readData());
        system("cls");
        printf("\n\n1 - ввести ещё\n0 - закончить ввод\n-> ");
        ch = getch();
        printf("\n");
        if (ch != '1' && ch != '0') ch = '0';
    } while (ch == '1');
    return top;
}

struct tree *addNode(struct tree *top, struct scientist data) {
    if (!top) {
        top = (struct tree*)malloc(sizeof(struct tree));
        if (!top) {
            printf("Ошибка выделения памяти!\n");
            return NULL;
        }
        top->info = data;
        top->left = top->right = NULL;
    } else if (data.articles <= top->info.articles) {
        top->left = addNode(top->left, data);
    } else /*(data.articles > top->info.articles)*/ {
        top->right = addNode(top->right, data);
    }
    return top;
}

void viewTree(struct tree *top) {
    if (top) {
        viewTree(top->left);
        printNode(top);
        viewTree(top->right);
    }
}

void printTreeStructure(struct tree *node, int space, int indent) {
    if (node == NULL) return;
    space += indent;
    printTreeStructure(node->right, space, indent);
    printf("\n");
    for (int i = indent; i < space; i++) printf(" ");
    printf("%d (%s) [статей: %d]\n", node->info.id, node->info.name, node->info.articles);
    printTreeStructure(node->left, space, indent);
}

struct tree *findMin(struct tree *node) {
    while (node && node->left) node = node->left;
    return node;
}

struct tree *deleteNode(struct tree *node, int articles) {
    if (node == NULL) {

```

```

        printf("Учёный с количеством статей %d не найден.\n", articles);
        return NULL;
    }
    if (articles < node->info.articles) {
        node->left = deleteNode(node->left, articles);
    } else if (articles > node->info.articles) {
        node->right = deleteNode(node->right, articles);
    } else {
        // нашли
        if (node->left == NULL) {
            struct tree *temp = node->right;
            free(node);
            return temp;
        } else if (node->right == NULL) {
            struct tree *temp = node->left;
            free(node);
            return temp;
        }
        struct tree *minNode = findMin(node->right);
        node->info = minNode->info;
        node->right = deleteNode(node->right, minNode->info.articles);
    }
    return node;
}

// Сохранение: pre-order с маркерами 'N' (NULL)
void saveToFile(struct tree *node, FILE *f) {
    if (node == NULL) {
        fprintf(f, "N\n");
        return;
    }
    fprintf(f, "D\n");
    fprintf(f, "%d\n", node->info.id);
    fprintf(f, "%s\n", node->info.name);
    fprintf(f, "%s\n", node->info.area);
    fprintf(f, "%s\n", node->info.degree);
    fprintf(f, "%d\n", node->info.articles);
    fprintf(f, "%d\n", node->info.quotes);
    fprintf(f, "%d\n", node->info.hirshIndex);
    saveToFile(node->left, f);
    saveToFile(node->right, f);
}

// Загрузка: чтение pre-order
struct tree *loadFromFile(FILE *f) {
    char buf[10];
    if (!fgets(buf, sizeof(buf), f)) return NULL;
    if (buf[0] == 'N') {
        return NULL;
    } else if (buf[0] == 'D') {
        struct scientist data;
        if (fscanf(f, "%d\n", &data.id) != 1) return NULL;
        if (fgets(data.name, sizeof(data.name), f) == NULL) return NULL;
        data.name[strcspn(data.name, "\n")] = '\0';
        if (fgets(data.area, sizeof(data.area), f) == NULL) return NULL;
        data.area[strcspn(data.area, "\n")] = '\0';
        if (fgets(data.degree, sizeof(data.degree), f) == NULL) return NULL;
        data.degree[strcspn(data.degree, "\n")] = '\0';
        if (fscanf(f, "%d\n", &data.articles) != 1) return NULL;
        if (fscanf(f, "%d\n", &data.quotes) != 1) return NULL;
        if (fscanf(f, "%d\n", &data.hirshIndex) != 1) return NULL;
        struct tree *node = (struct tree*)malloc(sizeof(struct tree));
        if (!node) return NULL;
        node->info = data;
    }
}

```

```

        node->left = loadFromFile(f);
        node->right = loadFromFile(f);
        return node;
    }
    return NULL;
}

int findRightmostLevel(struct tree *node) {
    int level = 1;
    while (node->right) {
        node = node->right;
        level++;
    }
    return level;
}

void printScientistsWithMoreThan100Articles(struct tree *node, int *count) {
    if (node) {
        printScientistsWithMoreThan100Articles(node->left, count);
        if (node->info.articles > 100) {
            printNode(node);
            (*count)++;
        }
        printScientistsWithMoreThan100Articles(node->right, count);
    }
}

void freeTree(struct tree *node) {
    if (node) {
        freeTree(node->left);
        freeTree(node->right);
        free(node);
    }
}

```

На рисунках 5.12 5.13 отображены результаты работы кода согласно всем условиям по заданию для языка C.

На рисунке 5.12 показан интерфейс меню.

На рисунке 5.13 показан процесс чтения дерева из текстового файла. Выбираем пункт меню 7, далее программа просит ввести название файла, после чего читает все данные из файла и выводит сообщение об успешно прочитанном дереве.

На рисунке 5.14 представлено содержание файла «to-test.txt».

На рисунке 5.15 представлено выполнение пункта меню 2 (просмотр дерева). В программе просмотр реализован обходом сверху-вниз. Данные представлены в виде таблицы для более лёгкого восприятия информации.

На рисунке 5.16 показан результат работы функции печати структуры дерева. Структура печатается с поворотом влево. Также каждая запись оформлена минимальным набором данных: ID, ФИО и кол-во научных статей.

На рисунке 5.17 показан результат поиска учёных с количеством статей > 100. Вывод стилизован в виде таблицы, для удобной работы с данными.

На рисунке 5.18 показан процесс добавления новой записи (узла) в дерево. Программа просит ввести все данные, затем сама автоматически ставит запись на нужное место в дереве, не нарушая его структуру.

На рисунке 5.19 показан процесс сохранения дерева в текстовый файл. В данном случае сохранение производится в файл «my-tree.txt».

На рисунке 5.20 показано содержимое файла «my-tree.txt» после сохранения дерева.

```
===== МЕНЮ =====  
1. Организация дерева  
2. Просмотр дерева (обход)  
3. Отображение структуры дерева  
4. Добавить узел дерева  
5. Исключить узел из дерева  
6. Сохранить дерево в файл  
7. Загрузить дерево из файла  
8. Определить уровень самого правого узла  
9. Вывести учёных с более чем 100 статей  
ESC. Выход  
-> |
```

Рисунок 5.12 – Интерфейс меню

```
Введите имя файла для загрузки: to-test.txt  
Дерево успешно загружено из файла to-test.txt  
Нажмите любую клавишу...|
```

Рисунок 5.13– Процесс чтения дерева из текстового файла

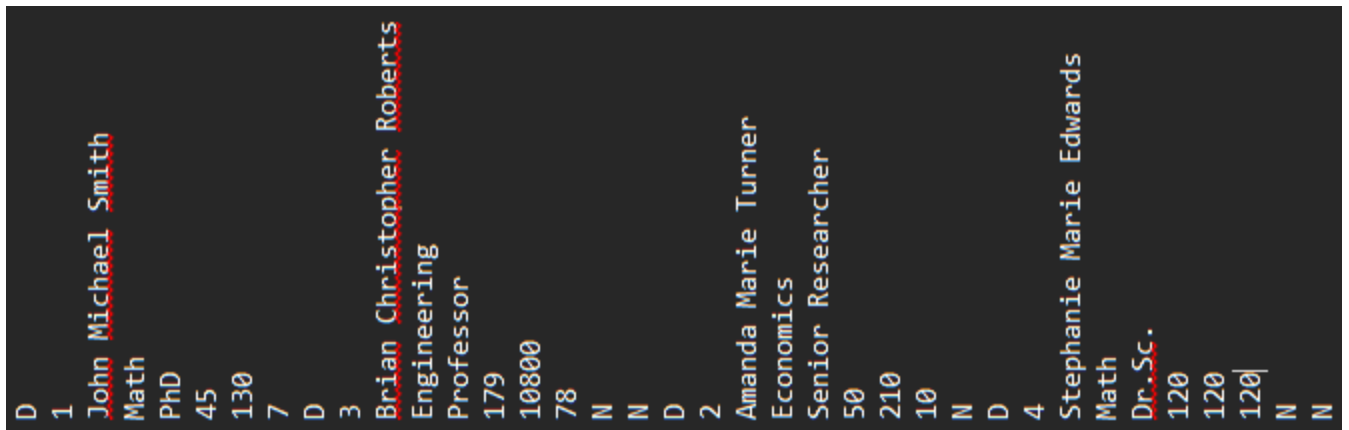


Рисунок 5.14 – Содержание файла to-test.txt

ID	Фамилия Имя Отчество	Учёная Степень	Область Науки	Кол-во статей	Кол-во цитирований	Индекс Хирша
3	Brian Christopher Roberts	Professor	Engineering	179	10800	78
1	John Michael Smith	PhD	Math	45	130	7
2	Amanda Marie Turner	Senior Researcher	Economics	50	210	10
4	Stephanie Marie Edwards	Dr.Sc.	Math	120	120	120

Нажмите любую клавишу...

Рисунок 5.15 – Результат обхода дерева сверху-вниз

```

Структура дерева (повёрнута влево):

      4 (Stephanie Marie Edwards) [статей: 120]
    2 (Amanda Marie Turner) [статей: 50]
  1 (John Michael Smith) [статей: 45]
    3 (Brian Christopher Roberts) [статей: 179]

Нажмите любую клавишу...|

```

Рисунок 5.16 – Результат отображения структуры дерева

Учёные с количеством статей > 100:

3	Brian Christopher Roberts	Professor	Engineering	179	10800	78
4	Stephanie Marie Edwards	Dr.Sc.	Math	120	120	120

Нажмите любую клавишу...|

Рисунок 5.17 – Результат поиска учёных с количеством статей более 100

```

Введите ФИО: Kevin Michael Mitchell
Введите научную область: Chemistry
Введите учёную степень: Senior Researcher
Введите количество научных статей: 72
Введите количество цитирований: 2270
Введите индекс Хирша: 23

Узел добавлен.
Нажмите любую клавишу...|

```

Рисунок 5.18 – Результат добавления нового узла в дерево

```
-> Введите имя файла для сохранения: my-tree.txt
Дерево сохранено в файл my-tree.txt
Нажмите любую клавишу...|
```

Рисунок 5.19 – Процесс сохранения дерева в текстовый файл

```
D 1 John Michael Smith
Math
PhD
45
130
7
D 3 Brian Christopher Roberts
Engineering
Professor
179
10800
78
N N D 2 Amanda Marie Turner
Economics
Senior Researcher
50
210
10
N D 4 Stephanie Marie Edwards
Math
Dr.Sc.
120
120
120
D 2 Kevin Michael Mitchell
Chemistry
Senior Researcher
72
2270
23
N N N
```

Рисунок 5.20 – Содержимое текстового файла my-tree.txt

ВЫВОД

В ходе лабораторной работы были исследованы особенности представления и обработки структур бинарных деревьев в языках C/C++. На практике были успешно реализованы алгоритмы работы с бинарными деревьями, включая их создание, обработку, удаление элементов, поиск элементов, чтение и сохранение в текстовый файл, а также просмотр структуры дерева. Экспериментально подтверждена и изучена неразрывная связь между указателями в теме бинарных деревьев в языках C/C++.